

ÇİP TASARIM YARIŞMASI
MİKRODENETLEYİCİ TASARIM KATEGORİSİ
ÖN TASARIM RAPORU ŞABLONU

TAKIM ADI: BLogic Mikroelektronik
BAŞVURU ID: 772759

2026



1. Giriş

BLogic Mikroelektronik takımı olarak 2026 TEKNOFEST Çip Tasarım Yarışması Mikrodenetleyici Tasarım Kategorisi kapsamında temel hedefimiz, 32-bit 4 aşamalı boru hattına sahip açık kaynaklı CV32E40P RISC-V işlemci çekirdeğini merkeze alarak; AXI4/AXI4-Lite veri yolu mimarisiyle haberleşen, çevre birimlerine ve bellek bloklarına sahip, uçtan uca doğrulanmış bir çip (SoC) tasarlamaktır. Projemizin şartname gereksinimlerine uygun olarak tasarlanacak TFLite Micro Speech modelini donanımsal seviyede gerçekleyecek olan bu hızlandırıcı; kendine ait 30 kB bellek alanı ve özel UART-stream arayüzü ile çalışarak, ses verilerini yüksek hızda sınıflandırmayı amaçlamaktadır. Sistemin genelinde FPGA prototiplemesinden başlayıp aşama aşama ilerleyerek üretime hazır GDSII çıktısına (ASIC akışı) kadar uzanan sağlam bir fiziksel tasarım süreci yürütülecektir. Belirlenen bu amaçlar doğrultusunda raporumuz üç ana bölümden oluşmaktadır. "Sistem Mimarisi" bölümünde, işlemci, bellek ve çevre birimleri arasındaki veri akışları top-level blok diyagramlarla aktarılacaktır. "Tasarım Detayları" bölümünde QSPI flash boot süreci, donanım arayüzleri, YZ hızlandırıcının çalışma mekanizması ve UVM destekli kapsamlı doğrulama stratejilerimiz açıklanacaktır. "Takım Organizasyonu ve İş Planı" kısmında ise ekip üyelerinin görev dağılımları, öngörülen takvim ve projenin başarıya ulaşması için belirlenen risk yönetimi planları detaylandırılarak rapor bütünlüğü sağlanacaktır.

2. Sistem Mimarisi

Bu proje kapsamında tasarlanacak sistem, 32-bit RISC-V mimarisi temel alınarak geliştirilecek, uçta yapay zeka (Edge AI) uygulamalarına yönelik özelleştirilmiş bir mikrodenetleyicidir. Sistemin işlem biriminin kalbinde, açık kaynaklı ve 4 aşamalı boru hattına (pipeline) sahip CV32E40P işlemci çekirdeği yer almaktadır. Bu mikrodenetleyicinin en büyük ayırt edici özelliği, ses sınıflandırma işlemlerini (TFLite Micro Speech modeli) donanımsal olarak ivmelendiren ve doğrudan veri akışı (stream) alabilen özel bir Yapay Zeka Hızlandırıcı barındırmasıdır.

Mikrodenetleyici içerisindeki modüller arası haberleşme, endüstri standardı olan AXI4 ve çevre birimleri için AXI4-Lite veriyolu (bus) mimarileri üzerinden sağlanmaktadır. Sistem, işlemcinin ve hızlandırıcının ihtiyaç duyduğu dahili SRAM bellek bloklarını (Buyruk, Veri ve YZ bellekleri) bünyesinde barındırmaktadır. Dış dünya ile iletişim, sensör kontrolleri ve zamanlama görevleri ise AXI-Lite köprüsüne bağlı GPIO, Timer, I2C, QSPI ve UART gibi standart çevre birimleri ile yönetilmektedir. Sistem gücü aldığı anda, QSPI arayüzü aracılığıyla harici bir flaş bellekten boot sürecini başlatarak ana programı dahili buyruk belleğine aktaracak otonom bir yapıya sahiptir.

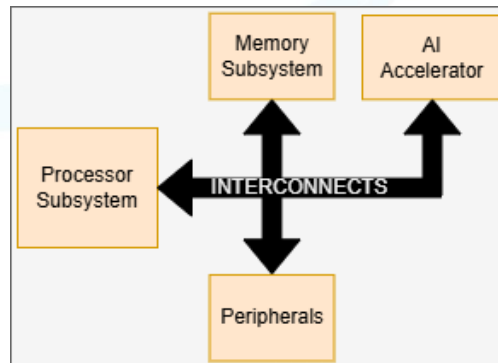


Fig.2.1 – Genel sistem mimarisi

Ostim Blogic Microelektronik	Ana Bloklar	Modüller	Açıklamalar
	Process Subsystem	* 32-bit CV32E40P RISC-V Çekirdeği * OBI-to-AXI4 Köprüsü * JTAG Arayüzü	Sistemin ana kontrol merkezidir. Bu blok içerisinde 32-bit RISC-V mimarisine sahip CV32E40P işlemci çekirdeği, çekirdeğin kendi yerel dili olan OBI protokolünü sistemin ana veri yoluna çeviren "OBI-to-AXI4 Köprüsü" ve sistem üzerinde hata ayıklama (debug) yapılmasını sağlayan opsiyonel JTAG arayüzü bulunmaktadır.
	Memory Subsystem	* Boot ROM * Buyruk (Instruction) SRAM * Veri (Data) SRAM * YZ (AI) SRAM	Mikrodenetleyicinin kod ve veri saklama ihtiyaçlarını karşılayan bloktur. - Sistem ilk açıldığında bootloader kodunun çalıştığı Boot ROM. - İşlemcinin çalıştıracağı kodların tutulduğu Buyruk (Instruction) SRAM. - Değişkenlerin ve genel verilerin tutulduğu Veri (Data) SRAM. - Sadece yapay zeka hızlandırıcısına hizmet eden YZ (AI) SRAM birimlerinden oluşmaktadır.
	(Bus & Interconnect)	* AXI4 Interconnect * AXI4-to-AXI4-Lite Köprüsü	Sistemdeki tüm modüllerin birbiriyle haberleşmesini sağlayan ana omurgadır. Yüksek hızlı bellek erişimleri ve veri transferleri için AXI4 Interconnect; düşük hızlı çevre birimlerinin konfigürasyon ayarları için ise AXI4-to-AXI4-Lite Köprüsü bu bloğu oluşturur.
	Peripherals	* GPIO (32-pin) * Timer * UART_0 ve UART_1 Stream * I2C Master * QSPI Master	Sistemin dış dünya ile etkileşime girmesini ve zamanlama işlemlerini yönetmesini sağlayan AXI-Lite uyumlu donanım modülleridir. Bu blok içerisinde; Dış dünya donanım kontrolleri için GPIO (32-pin), zamanlama için Timer, genel haberleşme ve veri yeri akışı için 2 adet UART, sensör okumaları için I2C Master ve sistemin harici flaş bellekten boot edilmesini sağlayan QSPI Master birimleri yer almaktadır.
	AI Accelerator	* TFLite Micro Speech Donanım Hızlandırıcısı	Ses sınıflandırma (TFLite Micro Speech) modelini donanımsal olarak çalıştıran özel bloktur. Yüksek hızlı bellek okuma/yazma işlemleri için AXI4 Master portuna, işlemci tarafından konfigüre edilebilmek için AXI4-Lite Slave portuna ve işlem bittiğinde işlemciyi uyarmak için bir donanımsal kesme (Interrupt) hattına sahiptir.

Tablo.2.1 - Tasarlanacak sistemin genel tanımı ve özellikleri

Kullanılacak tasarım ve doğrulama araçları, proje geliştirme süreçlerimizin tamamı WSL ortamında yürütülecektir.

- İşlemci Çekirdeği (CV32E40P RISC-V):** Çekirdek doğrulaması ve komut izlerinin (instruction trace) eşleşme kontrolleri için Spike ISS kullanılacaktır. Mikrodenetleyici üzerinde koşacak yazılımların derlenmesi için RISC-V GNU Compiler Toolchain tercih edilmiştir.
- Sistem Veriyolu ve Çevre Birimleri:** AXI protokol kontrolleri ve çevre birimi doğrulamaları, SystemVerilog ve UVM kullanılarak gerçekleştirilecektir. Simülatör olarak UVM desteği sunan açık kaynaklı Verilator kullanılacaktır.
- Yapay Zeka Hızlandırıcı:** TFLite Micro Speech modelinin RTL gerçeklemesi SystemVerilog ile tasarlanacaktır. Modelin eğitimi, test verilerinin oluşturulması ve yazılım referans modelinin analizi için Python ve TensorFlow Lite ortamlarından faydalanılacaktır.
- FPGA Prototipleme ve Sentez:** Tasarımın FPGA geliştirme kartı üzerinde doğrulanması ve bitstream üretimi için Xilinx Vivado araç zinciri kullanılacaktır.
- Fiziksel Tasarım:** Tasarımın üretime hazır GDSII formatına getirilmesi, sentez, P&R (Place & Route) ve statik tasarım kontrolleri (LVS, DRC, vb.) için komite tarafından sağlanacak olan Cadence, Synopsys veya Siemens EDA araçları kullanılacaktır.
- Otomasyon ve Sürüm Kontrolü:** Bütün simülasyon, sentez ve implementasyon akışlarının jüri tarafından tek tıkla koşturulabilmesi amacıyla Makefile ve Python script'leri geliştirilecektir. Sürüm kontrolü ve proje takibi için GitHub kullanılacaktır.

Performans Metriği	YZ Çıkarım Doğruluğu	YZ İşlem İvmelendirmesi	Veri İşleme Hızı	Hedef Frekans	Çip Alanı ve Güç
Hedeflenen Değer	<%10 Sapma	> 1.0x (Yazılımdan Hızlı)	Yüksek (Cycle başına)	~50 MHz (FPGA)	Minimum
Açıklama	TFLite Micro Speech yazılım referans modelinin sonuçlarına %10'luk doğruluk penceresi içinde yaklaşılması.	RISC-V çekirdeği üzerinde çalışan yazılım gerçekleştirilmesine kıyasla daha yüksek hızlanma elde edilmesi.	Clock cycle ve saniye bazında işlenen veri miktarının maksimize edilmesi.	Sistemin ses veri akışını gerçek zamanlı işleyebilmesi için hedeflenen minimum saat frekansı.	Uçta yapay zeka konseptine uygun olarak düşük alan (gate count) ve düşük dinamik güç tüketimi hedeflenmesi.

3. Tasarım Detayları

Tasarladığımız sistem, 32-bit RISC-V mimarisine sahip CV32E40P işlemci çekirdeği etrafında şekillendirilmiştir. Çekirdeğin yerel OBI protokolü, yüksek hızlı bellek ve YZ hızlandırıcı erişimleri için "OBI-to-AXI4" köprüsüyle ana AXI4 omurgasına; düşük hızlı çevre birimlerinin yönetimi içinse "AXI4-to-AXI4 Lite" köprüsüyle ikinci bir veriyoluna bağlanır. Bellek mimarimiz; 8 KB Buyruk, 8 KB Veri, YZ işlemlerine özel 30 KB AI SRAM ve 1 KB Boot ROM'dan oluşmaktadır. Enerjilendirme anında Boot ROM'daki bootloader, QSPI Master aracılığıyla harici flash bellekten programı okuyup Buyruk belleğine kopyalar ve yürütmeyi buraya devreder. TFLite Micro Speech modelini RTL düzeyinde koşturan donanım hızlandırıcımız, konfigürasyon için AXI4-Lite Slave, veri erişimi için AXI4 Master portunu kullanır. Gerekli verileri özel "UART_1 Stream" üzerinden alıp AI SRAM'e yazar ve çıkartım işlemi bittiğinde işlemciye donanımsal kesme (interrupt) gönderir. AXI4-Lite veriyoluna bağlı çevre birimlerimiz ise 32-pin GPIO, Timer, genel ve veri akışı amaçlı iki adet UART, I2C Master, QSPI Master ile hata ayıklama için eklenen opsiyonel JTAG arayüzünden oluşmaktadır.

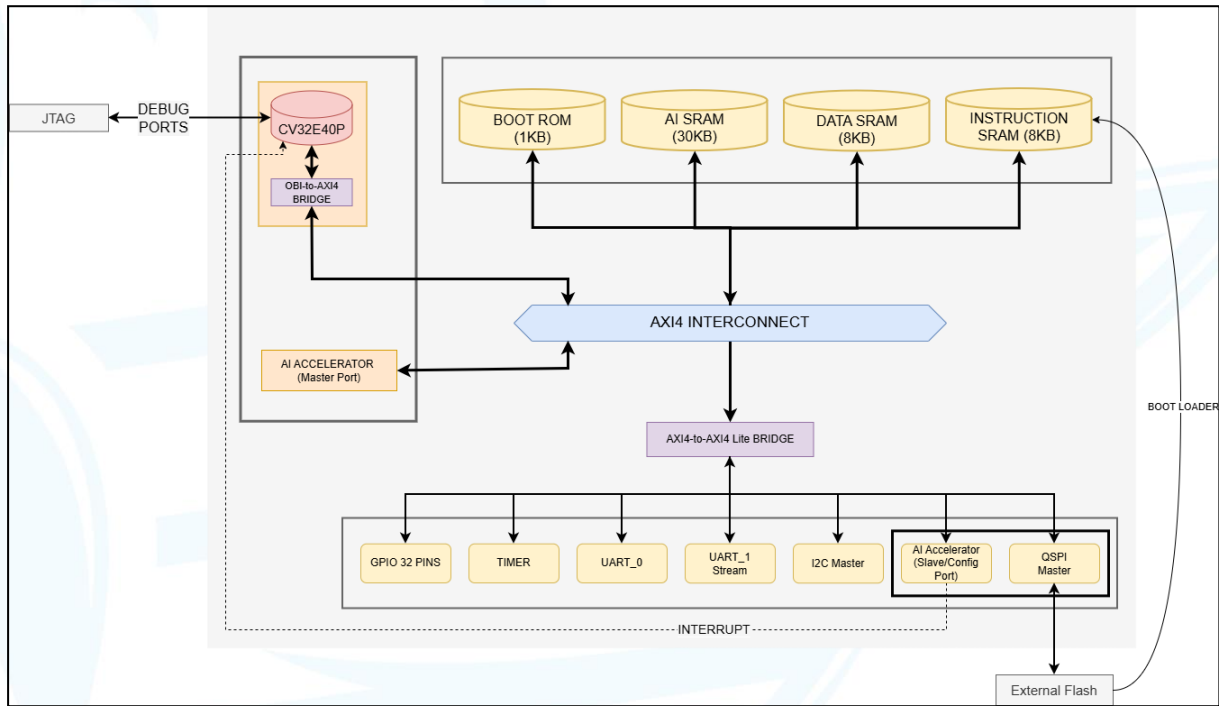


Fig.3.1 – Detaylı blok diyagramı

Yarışma da kullanacağımız CV32E40P, gömülü sistemler ve mikrodenetleyici tasarımları için tasarlanmış, 32-bit mimariye sahip, tek çekirdekli ve 4-stage pipeline'a sahip bir RISC-V işlemcidir. Çekirdek, standart RV32IM[F|Zfinx]C instruction setlerini destekleyecek şekilde konfigüre edilebilir yapıdadır. Sistemimizde çekirdeğin çalışması için 8 kB instruction ve 8 kB data belleği sisteme dâhil edilmiştir. Çekirdeğin kendi doğal bellek arayüzü olan OBI (OpenBus Interface), sistemin geri kalanıyla haberleşebilmesi için bir köprü mimarisi gerektirmektedir. CV32E40P işlemcisinin OBI portu, bir OBI-to-AXI4 Bridge vasıtasıyla ana AXI4 ara interconnect'e bağlanmaktadır. Çekirdek; donanım hızlandırıcılara, çevre birimlerine (UART, SPI, Timer vb.) ve bellek birimlerine bu standart AXI4 arayüzü üzerinden erişim sağlamaktadır.

3.1 - RISC-V çekirdeğin doğrulaması

Çekirdeğin doğrulaması, Instruction Trace eşlemesi yöntemiyle ve kendi kendini kontrol eden bir mimaride gerçekleştirilecektir. Doğrulama sürecinde, RISC-V mimarisinin resmi komut seti simülatörü olan Spike ISS kullanılacaktır. Çekirdeğin SystemVerilog tabanlı tasarımı ise Verilator simülatörü üzerinde, geliştirilen testbench içerisinde çalıştırılacaktır. Yazılan test yazılımları, RISC-V GNU Toolchain ile derlendikten sonra eşzamanlı olarak hem Spike ISS üzerinde hem de Verilator'daki RTL model üzerinde çalıştırılacaktır. Her clock cycle da işlenen komutlar sonucunda ortaya çıkan Program Counter, genel amaçlı register değişimleri, Memory Load/Store adres ve bilgileri iki simülatörden log (trace) dosyaları olarak çekilecek ve karşılaştırılacaktır. Herhangi bir uyumsuzluk durumunda test hata verecek ve tasarımın hangi komutta veya veri yolunda saptığı tespit edilecektir.

Çekirdeğin doğrulanması için şartnamede belirtilen directed ve random test yaklaşımları bir arada kullanılacaktır. RV32IM[F/Zfinx]C instruction setini standartlara uygun işlediğini kanıtlamak için, açık kaynaklı riscv-arch-test [1] kullanılacak ve C dilinde yazılmış; özellikle yapay zeka hızlandırıcısı ile haberleşmeyi, bellek sınırlarını ve interrupt yönetimini hedefleyen yönlendirilmiş donanım/yazılım testleri uygulanacaktır. Tasarımın öngörülemez uç durumlarını yakalamak amacıyla, açık kaynaklı riscv-dv[2] kullanılacaktır.

Code Coverage bağlamında Verilator üzerinden alınacak raporlarla RTL kodunun satır (line), durum (toggle) ve dallanma (branch) bazında ne kadarının test edildiği ölçülecek ve test edilmeyen (dead code veya ulaşılamayan durumlar) kısımlar tespit edilecektir.

Functional Coverage bağlamında tasarımın kritik kontrol durumları için SystemVerilog Assertions (SVA) yazılarak, belirlenen işlevsel hedeflerin %100 oranında test edilip edilmediği denetlenecektir.

3.2 - Sistem doğrulaması

Tasarımımızın doğrulama süreci, hataları en erken aşamada yakalamak ve sistem kararlılığını garanti altına almak amacıyla sistem seviyesinde titizlikle yürütülecektir. Sistemin sıfırlama anından itibaren komut belleğinden doğru adresleri okuması ve veri belleğiyle iletişim kurması doğrulanacaktır. Sisteme entegre edilen UART, SPI ve Timer gibi çevre birimleri, veri yolundaki memory map'e uygun olarak directed C testleri ile sınanacaktır. Çevre birimlerinin FIFO sınır değerleri (full/empty) ve ürettikleri donanım kesmelerinin (IRQ) işlemci tarafından doğru önceliklendirmeye işlenmesi, manuel inceleme gerektirmeyen, kendi kendini kontrol eden test senaryolarıyla denetlenecektir.

SoC içerisindeki AXI ve çevre birimi haberleşme arayüzlerinde SystemVerilog Assertions (SVA) kullanılarak dinamik protokol kontrolleri uygulanacaktır. Örneğin; geçerli bir veri iletimi (handshake) sırasında hatta 'X' (tanımsız) veya 'Z' (yüksek empedans) verisi gelmesi, bekleme (wait) sürelerinin aşılması gibi olası protokol ihlalleri simülasyonlar sırasında anında yakalanarak raporlanacaktır.

Yapay zeka hızlandırıcısının entegrasyon testlerinde, TFLite Micro (Micro Speech) algoritmasının basitleştirilmiş bir modeli doğrudan mikrodenetleyici üzerinde çalıştırılacaktır. Sistem seviyesi veri akışı doğrulanırken örnek giriş verileri (ses örnekleri), tahsis edilen UART-stream arayüzü üzerinden alınarak doğrudan ilgili bellek alanına aktarılacak, işlemci, AXI-Lite

arayüzü üzerinden hızlandırıcının CSR (Control and Status Register) yazmaçlarını konfigüre ederek işlemi başlatacak ve hızlandırıcı, işlemciyi meşgul etmeden, işleyeceği verileri ana AXI arayüzü üzerinden bağımsız olarak bellekten çekecektir.

Hızlandırıcının hesabı tamamlandığında üretilen YZ çıktıları, Python tabanlı referans çıktılarıyla doğrudan C test kodu içerisinde beklenen referans değerlerle (golden data) kıyaslanacaktır. Bu "self-checking" (kendi kendini kontrol eden) yapı sayesinde sistemin doğruluğunu kanıtlamış olacaktır.

3.3 - UVM Doğrulama

Tasarımımızdaki karmaşık donanım bloklarının (özellikle AXI veri yolu, çevre birimleri ve yapay zeka hızlandırıcısı) işlevsel doğrulaması, endüstri standardı olan UVM ile yapılacaktır. Bu bağlamda, UVM Sequence yapıları kullanılarak çeşitli senaryolar oluşturulacaktır. AXI veri yoluna ve hızlandırıcı arayüzlerine manuel testlerle düşünülmesi zor olan rastgele fakat geçerli veri trafiği (farklı bekleme süreleri, ardışık okuma/yazma istekleri, rastgele burst boyutları) basılacaktır. Bu sayede tasarımın öngörülemez uç durumları otomatik olarak tetiklenecektir.

İşlemci henüz devrede olmadığı testlerde, arayüzlere dışarıdan sinyal süren Aktif UVM Ajanları kullanılacaktır. Sistem seviyesine (SoC) geçildiğinde ise bu ajanlar kolayca Pasif (Monitor) moda alınarak sadece veri yolunu dinleyen ve protokol kontrollerini yapan şekle geçecektir.

Tasarımdan (DUT) UVM Monitor'ler aracılığıyla toplanan çıktılar, UVM Scoreboard yapısına iletilecektir. Scoreboard, donanımdan gelen bu gerçek sonuçları, Python/C++ tabanlı referans modelin ürettiği beklenen sonuçlarla simülasyon anında anlık olarak karşılaştıracaktır. Bu otonom ve kendi kendini kontrol eden (self-checking) mimari sayesinde, tasarımın veri tutarlılığı insan müdahalesine gerek kalmadan doğrulanacaktır.

Üretilen rastgele senaryoların ve uygulanan testlerin tasarımın ne kadarını fiziksel olarak sınıadığı, UVM ortamına entegre edilecek SystemVerilog yapıları ile ölçülerek %100 kapsama hedefine ulaşılacaktır.

3.4 - Hızlandırıcı İçin Yapılacak Çalışmalar

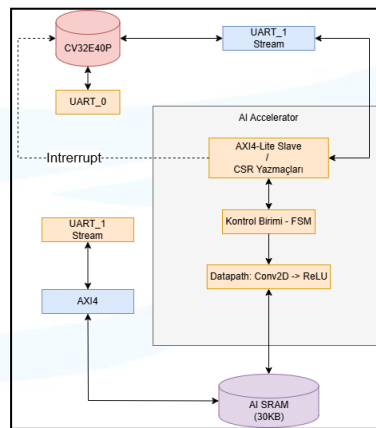


Fig.3.2 – Yapay zeka hızlandırıcı için planlanan donanım blokları

Yapay zeka hızlandırıcımız, ses verilerini sınıflandırmak amacıyla TFLite Micro Speech modelini RTL düzeyinde gerçekleyecektir. Bu doğrultuda "Tiny Conv" mimarisi temel alınarak, yazılım gerçeklemesine kıyasla acceleration sağlanması hedeflenmektedir.

Hızlandırıcı mimarimiz, modelin katmanlarını verimli bir şekilde işlemek üzere 4 ana birimden oluşacaktır. İlk olarak CSR yazmaçları kısmı, işlemcinin hızlandırıcıyı konfigüre etmesini, giriş/çıkış bellek adreslerini belirlemesini ve işlemi başlatmasını sağlayacaktır. İkinci olarak kontrol birimi (FSM) kısmı, tüm veri akışını ve işlem sırasını yöneten ana kontrol merkezi olup katmanlar arası geçişleri ve bellek erişim zamanlamalarını koordine edecektir. Üçüncü kısım modelin temelini oluşturan DepthwiseConv2D, ReLU, FullyConnected ve Softmax işlemlerini gerçekleştiren aritmetik birimlerden oluşacaktır. Son olarak AXI4 master arayüzü, hızlandırıcının işlemciden bağımsız olarak 30 KB'lık AI SRAM üzerinden veri okuma ve yazma işlemlerini yüksek hızda yapmasını sağlayacaktır.

Sistemdeki veri trafiği, şartnamede belirtilen otonom yapıyı destekleyecek şekilde yapılacaktır. İlk olarak çıkarım yapılacak ses verileri, özel UART_1 (Stream) arayüzü üzerinden sisteme akar ve doğrudan AI SRAM içerisindeki ilgili bölgeye yazılacaktır. Daha sonra, işlemci AXI4-Lite üzerinden hızlandırıcının CSR yazmaçlarına gerekli ayarları yapıp "Başlat" komutunu gönderecektir. Daha sonrasında ise hızlandırıcı, AXI4 Master portunu kullanarak verileri bellekten çekip datapath üzerinde Tiny Conv mimarisine göre (Evrışım -> ReLU -> Tam Bağlantılı) işleyip ve sonuçları tekrar belleğe yazacaktır ve son olarak işlem tamamlandığında hızlandırıcı bir donanımsal interrupt üretecek, işlemci interrupt service routine'ini çalıştırarak sonucu alacak ve UART_0 üzerinden dış dünyaya aktaracaktır.

Donanımın doğrulanması ve çalıştırılması için belli yazılım araçları ve süreçler ile ilerlenecektir. TensorFlow Lite ortamında modelin Python/C++ referans çıktıları oluşturulacaktır. Yazılan kontrol kodları ve sürücüler, RISC-V GNU Compiler Toolchain kullanılarak derlenecektir ve RTL simülasyonu (Verilator) sırasında, donanımdan gelen sonuçlar yazılım referans modeliyle otomatik olarak kıyaslanacak ve %10'luk doğruluk penceresi içerisinde kaldığı teyit edilecektir.

3.5 - Mikrodenetleyici Donanım Tasarımı Bittikten Sonra, Hazırlanan Yazılım Uygulamalarının Nasıl Çalıştırılacağı

Mikrodenetleyici donanım (RTL) tasarımının tamamlanmasının ardından; TFLite Micro (Micro Speech) yapay zeka algoritması, donanım sürücüler ve çevre birimi kontrol rutinlerini içeren C/C++ tabanlı yazılım uygulamaları donanım üzerinde dört temel aşamada koşturulacaktır:

İlk olarak yazılan C/C++ kodları, standart bir bilgisayar işlemcisi yerine doğrudan geliştirdiğimiz donanıma özgü makine kodlarına dönüştürülmek üzere RISC-V GNU Compiler Toolchain kullanılarak derlenecektir. Derleme sürecinin en kritik adımı olan bağlama (linking) aşamasında, sisteme özel hazırlanan bir Linker Script (.ld) dosyası kullanılacaktır. Bu dosya; derlenen komut kodlarını (.text bölümü) 8 KB'lık Buyruk Belleğine (Instruction RAM), değişkenleri ve yığın belleğini (.data, .bss, .stack) ise 8 KB'lık Veri Belleğine (Data RAM) donanımın adres haritasına (memory map) birebir uyacak şekilde yerleştirecektir. İkinci olarak Derleyici çıktısı olarak elde edilen .elf (Executable and Linkable Format) dosyası, doğrudan donanıma yüklenemeyeceği için hexadecimal'a (.hex veya .bin) dönüştürülecektir. Bu dosya, işlemcinin tam olarak anlayacağı saf makine komutlarını (opcode) ve başlangıç verilerini barındıracaktır. Üçüncü olarak oluşturulan .hex dosyası, hedeflenen platforma göre üç farklı yöntemle sisteme aktarılacaktır:

İlk olarak Spike ISS ortamında çalıştırılacaktır. Derleyici çıktısı olan .elf dosyası doğrudan simülatöre argüman olarak verilecek. Spike, .elf içindeki adres haritasını okuyarak kendi sanal belleğini oluşturacak ve referans komut izi (instruction trace) loglarını üretmek üzere yazılımı saniyeler içinde koşturacaktır. İkinci olarak Verilator simülasyon ortamında çalıştırılacaktır.

Dönüşümü olan .hex dosyası, SystemVerilog kodları kullanılarak RTL simülasyonuna aktarılacaktır. Simülasyonun başlangıcında bu komutlar doğrudan modellenen SRAM bloklarının içerisine yüklenerek memory initialization yapılacak ve donanım boot edilecektir. Son olarakta FPGA prototipleme ortamında çalıştırılacaktır. Bellek dosyası, Xilinx Vivado araçları kullanılarak doğrudan FPGA içindeki BRAM modüllerine sentez aşamasında gömülecek veya sistemdeki bir bootloader yardımıyla UART arayüzü üzerinden çalışma zamanında belleğe aktarılacaktır.

Son olarak memory initialization'dan sonra donanıma verilen "Reset" sinyali kaldırıldığında işlemci uyanacaktır. Program Counter, doğrudan mimaride tanımlı olan ilk sabit bellek adresine yönlenecektir. Sistem öncelikle C dilinin çalışabilmesi için gerekli olan assembly başlangıç kodunu çalıştıracaktır. Bu kod, Stack Pointer Veri RAM'inin en üst adresine ayarlayacak, .bss (sıfırlanmış değişkenler) bölgesini temizleyecek ve tüm bu donanım hazırlıkları bittikten sonra yazılımın ana fonksiyonu olan main() fonksiyonun bulunduğu yere jump yaparak TFLite Micro döngüsünü resmen başlatacaktır.

3.6 - Çip Tasarım ve Fiziksel Gerçekleme Akışı

Çip tasarım akışımız, RTL kodlama aşamasından üretime hazır GDSII formatının elde edilmesine kadar endüstri standartlarına uygun olarak komite tarafından sağlanacak ticari EDA araçları (Synopsys, Cadence veya Siemens) üzerinden gerçekleştirilecektir. Süreç, donanım mimarisinin SystemVerilog ile tasarlanması ve fonksiyonel doğrulamalarının tamamlanmasıyla başlar. Ardından "Mantıksal Sentez (Logic Synthesis)" aşamasında RTL kodlarımız, hedef üretim teknolojisine (PDK) uygun mantık kapılarına dönüştürülerek kapı seviyesi ağ listesi (gate-level netlist) elde edilir. Sentez sonrası, tasarımın hedef frekansları karşıladığını doğrulamak için Statik Zamanlama Analizi (STA) uygulanır. Fiziksel tasarım (PnR) süreci ise sırasıyla; çip alanının belirlenip güç/toprak ağlarının (power mesh) oluşturulduğu Kat Planlama (Floorplanning), standart hücrelerin optimum şekilde yerleştirilmesi (Placement), saat sinyalinin çip geneline minimum sapmayla dağıtılmasını sağlayan Saat Ağacı Sentezi (CTS - Clock Tree Synthesis) ve son olarak hücreler arası sinyal bağlantılarının çizildiği Yönlendirme (Routing) adımlarından oluşur. Akışın son aşamasında ise tasarımın üretilebilirliğini garanti altına almak amacıyla LVS, DRC ve ERC gibi fiziksel doğrulama (Sign-off) testleri başarıyla koşularak fabrikaya gönderilecek nihai LEF, DEF ve GDSII çıktıları oluşturulacaktır. Tüm bu implementasyon akışı, otomasyonu ve jüri tarafından tekrar edilebilirliği sağlamak adına Python, Shell, TCL ve Makefile script'leri ile belgelendirilecek ve dokümente edilmiş bir şekilde GitHub üzerinden paylaşılacaktır.

4. Takım Organizasyonu ve İş Planı

Berk Muammer KUZU: Ostim Teknik Üniversitesi, Elektrik-Elektronik Mühendisliği, 3. Sınıf.

Sorumlulukları: YZ Hızlandırıcı ve çevre birimlerinin RTL (SystemVerilog) tasarımı, AXI4/AXI4-Lite veriyolu entegrasyonu, FPGA sentezi ve Fiziksel Tasarım (PnR, GDSII çıktıları) süreçleri.

Berkin Demircan: Ostim Teknik Üniversitesi, Bilgisayar Mühendisliği, 3. Sınıf.

Sorumlulukları: CV32E40P çekirdeğinin Spike ISS ile doğrulanması, UVM/Verilator tabanlı test senaryolarının (SVA, agent) yazılması, bootloader ve çevre birimi test yazılımlarının (firmware) geliştirilmesi.

Ostim Blogic Microelektronik	Yapılacak İş	Sorumlu Kişi	Öngörülen Zaman / Bitiş Hedefi
	Sistem Mimarisi ve ÖTR Hazırlığı: Mimari kararların alınması, blok diyagramların çizimi ve Ön Tasarım Raporu teslimi.	Berk & Berkin	4 Hafta (30 Saat) 16 Mart 2026
	Alt Blok RTL Tasarımları: Çevre birimlerinin ve YZ Hızlandırıcının (TFLite Micro) SystemVerilog ile kodlanması.	Berk	5 Hafta (45 Saat) 20 Nisan 2026
	Çekirdek ve Yazılım Geliştirme: Spike ISS entegrasyonu, C/Assembly test kodları ve bootloader yazımı.	Berkin	5 Hafta (45 Saat) 20 Nisan 2026
	Sistem Entegrasyonu (SoC) ve Doğrulama: AXI4 veriyolu bağlantılarının yapılması ve UVM tabanlı protokol/kapsam (coverage) testleri.	Berk & Berkin	4 Hafta (50 Saat) 10 Mayıs 2026
	DTR (Detay Tasarım Raporu) Hazırlığı: Simülasyon sonuçlarının derlenmesi ve detaylı rapor teslimi.	Berk & Berkin	2 Hafta (20 Saat) 15 Mayıs 2026
	FPGA Prototipleme ve Debug: Tasarımın FPGA'e gömülmesi, bitstream eldesi ve donanım üzerinde gerçek zamanlı testler.	Berk & Berkin	4 Hafta (40 Saat) 15 Haziran 2026
	Fiziksel Tasarım (Çip Akışı): Ticari EDA araçları ile Sentez, Floorplan, CTS, PnR ve DRC/LVS fiziksel sign-off testleri.	Berk	6 Hafta (50 Saat) 25 Temmuz 2026
	Final Teslimleri ve Demo Hazırlığı: GDSII dosyalarının ve kodların yüklenmesi, sunum hazırlığı.	Berk & Berkin	1 Hafta (15 Saat) 31 Temmuz 2026

5. Kaynakça ve Ekler

- [1] RISC-V International, "riscv-arch-test," [Çevrimiçi]. Erişim: <https://github.com/riscv/riscv-arch-test>
- [2] Google, "riscv-dv: RISC-V Instruction Generator," [Çevrimiçi]. Erişim: <https://github.com/chipsalliance/riscv-dv>
- [3] Arm Limited, "AMBA AXI and ACE Protocol Specification," Arm Developer Documentation. [Çevrimiçi]. Erişim: <https://developer.arm.com/documentation/ih0022/latest>
- [4] Advanced Micro Devices (AMD), Inc., "AXI Interface," Model Composer and System Generator User Guide. [Çevrimiçi]. Erişim: <https://docs.amd.com/r/en-US/ug1483-model-composer-sys-gen-user-guide/AXI-Interface>
- [5] A. P. Smitha, S. Ashwini, "UVM Based Test Bench to Verify AMBA AXI4 Slave Protocol," International Research Journal of Engineering and Technology (IRJET), vol. 3, no. 7, s. 1741, Tem. 2016. [Çevrimiçi]. Erişim: <https://www.irjet.net/archives/V3/i7/IRJET-V3I7327.pdf>
- [6] R. David vd., "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," in Proceedings of Machine Learning and Systems (MLSys), 2021. [Çevrimiçi]. Erişim: <https://proceedings.mlsys.org/paper/2021/file/6c44dc73014d66ba49b28d483a8f8b0d-Paper.pdf>
- [7] P. Warden ve D. Situnayake, TinyML, 1. Baskı, O'Reilly Media, 2019.
- [8] D. Harris ve S. Harris, Digital Design and Computer Architecture: RISC-V Edition, Morgan Kaufmann, 2021.
- [9] N. H. E. Weste ve D. Harris, CMOS VLSI Design, 4. Baskı, Pearson, 2010.
- [10] C. Spear, SystemVerilog for Verification, 2. Baskı, Springer, 2008.